Deletion in B-Tree

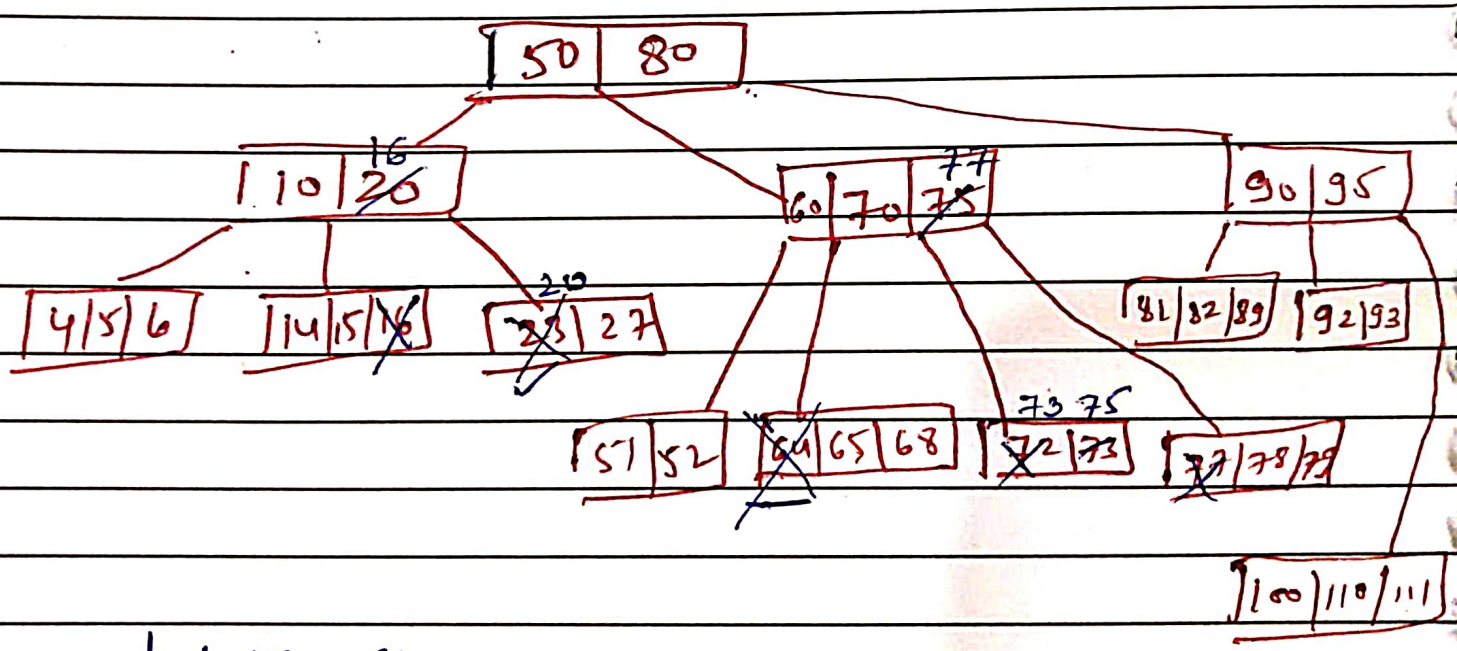
if order (m) = 5

min children = $\lceil m/2 \rceil = \lceil 5/2 \rceil = 3$

max. children = $m = 5$

min keys = $\lceil \frac{m}{2} \rceil - 1 = 2$

max keys = $m - 1 = 4$



delete 64

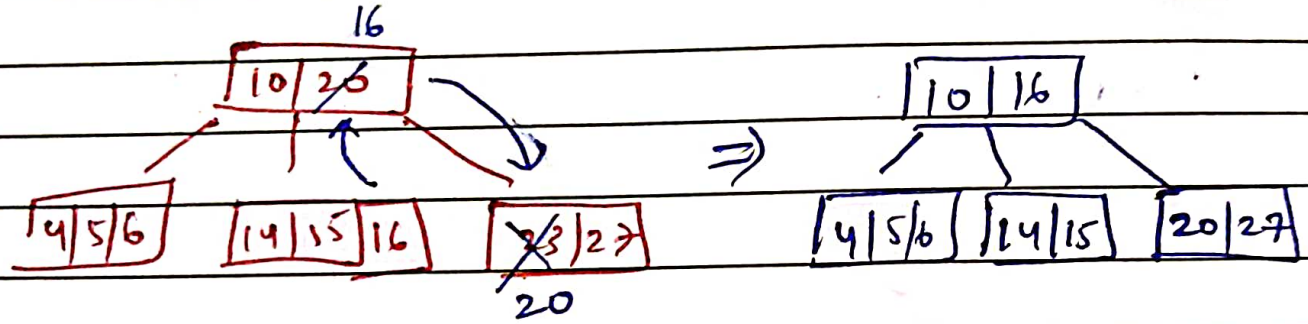
* target key is in leaf node & it contains more than min. no. of keys.
simply delete it.

delete 23

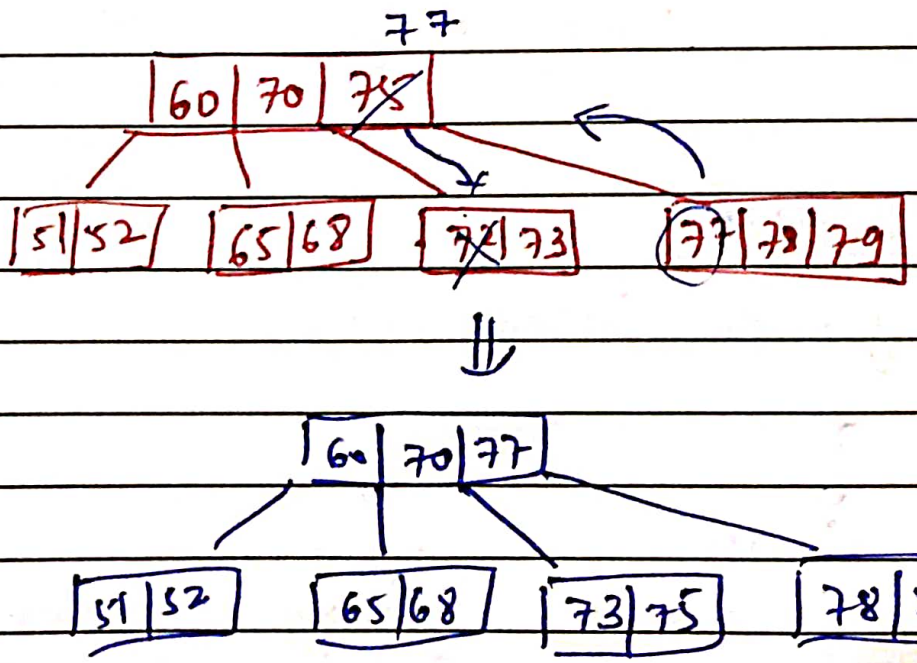
→ cannot simply delete 23 bcoz the node contains min. no. of keys.

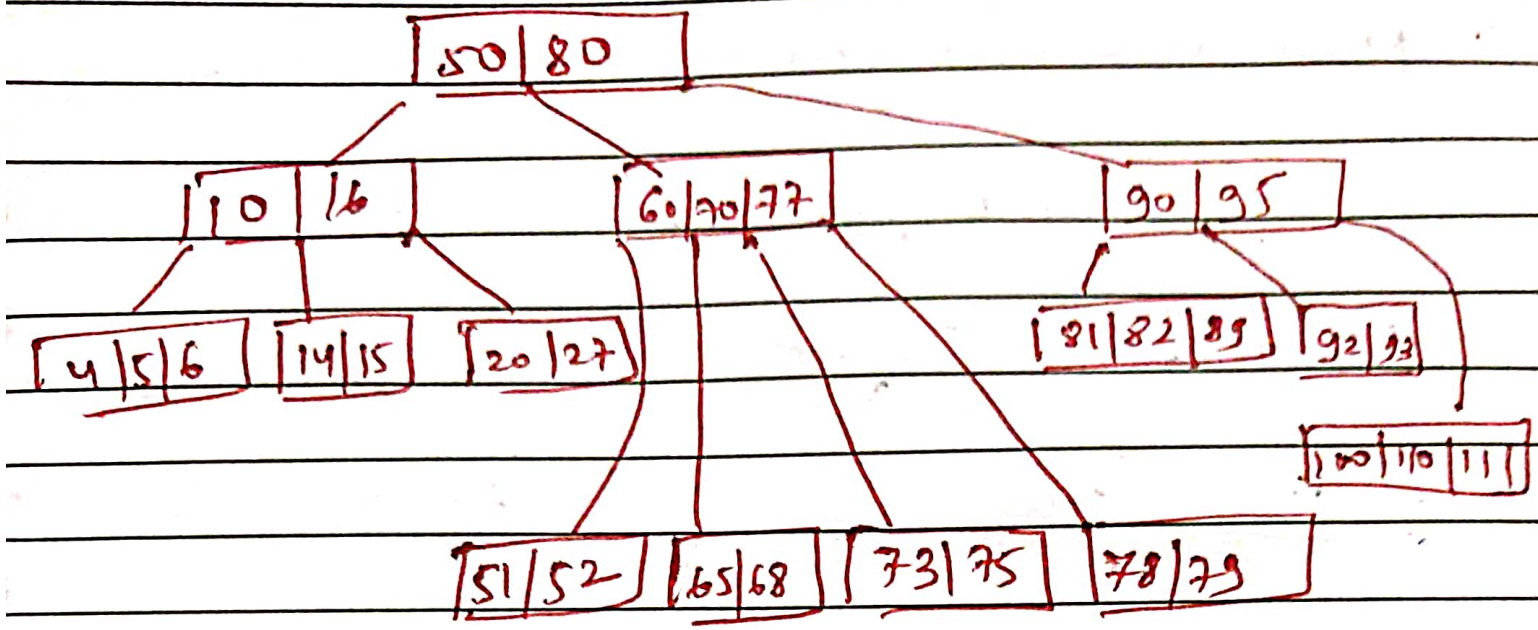
→ it will borrow from immediate left sibling

→ key will be borrowed through parent.



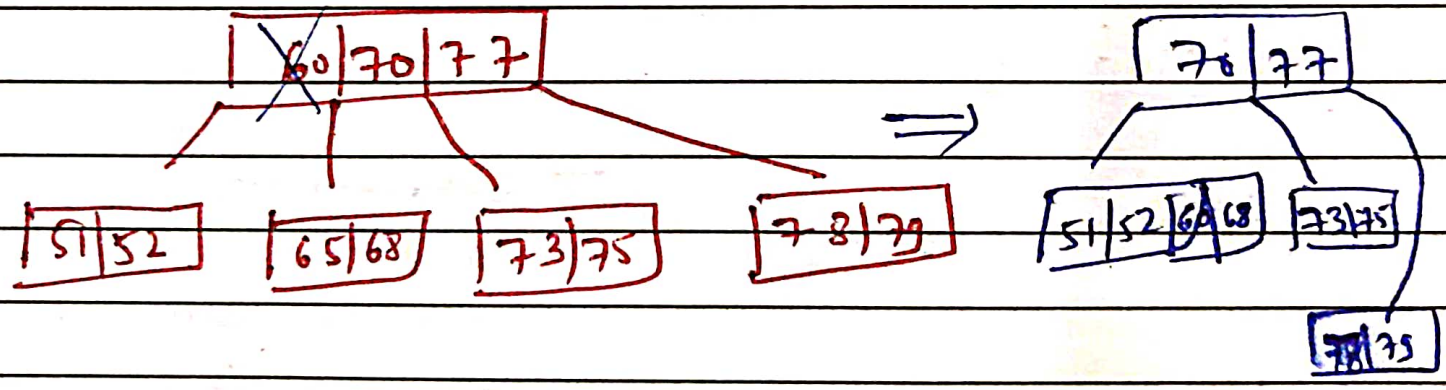
delete 72 → borrow from right sibling



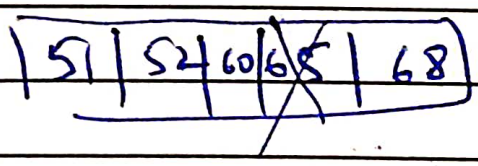


delete 65 → neither left sibling nor right sibling is having more than min no of keys.

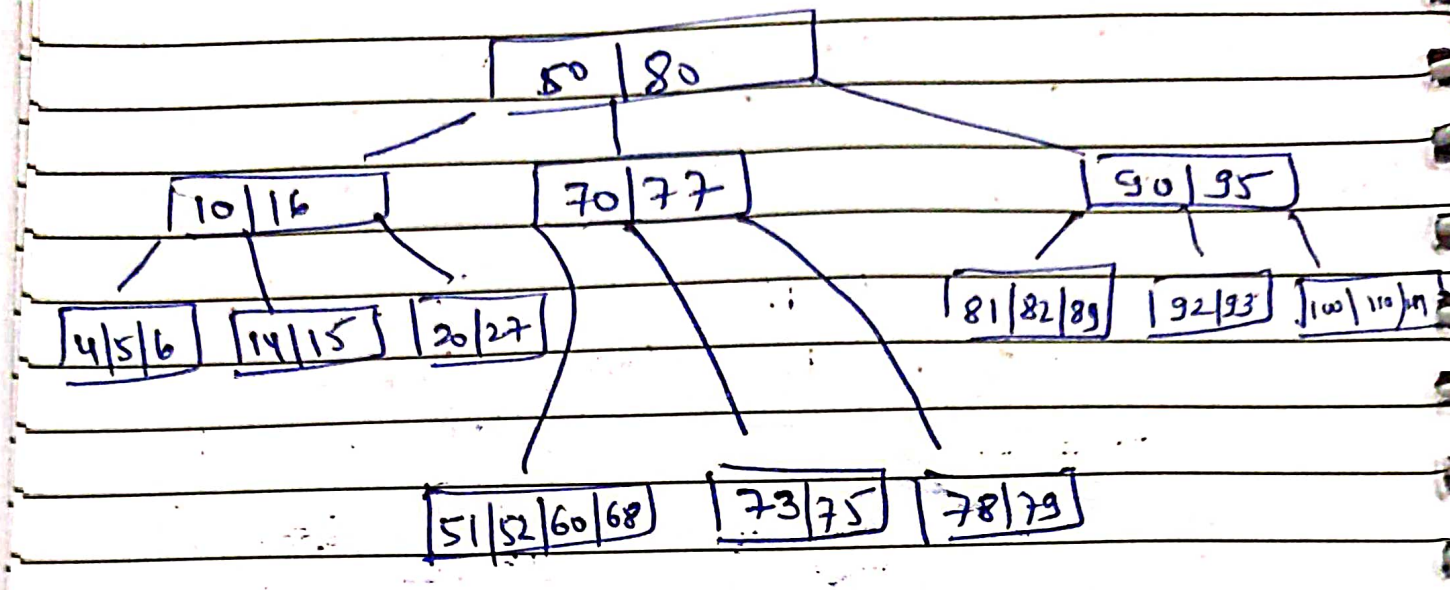
→ In this case, merging needs to be done either with left sibling or right sibling.



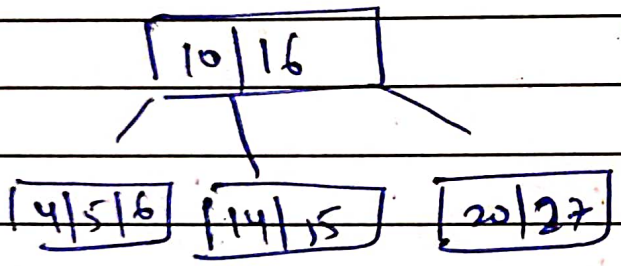
→ let's merging with left sibling & parent.



Now tree will become

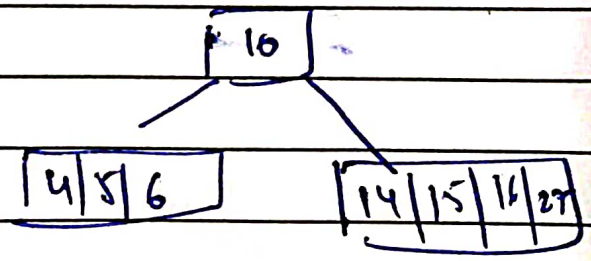


<u>delete 20</u> → merging will take place

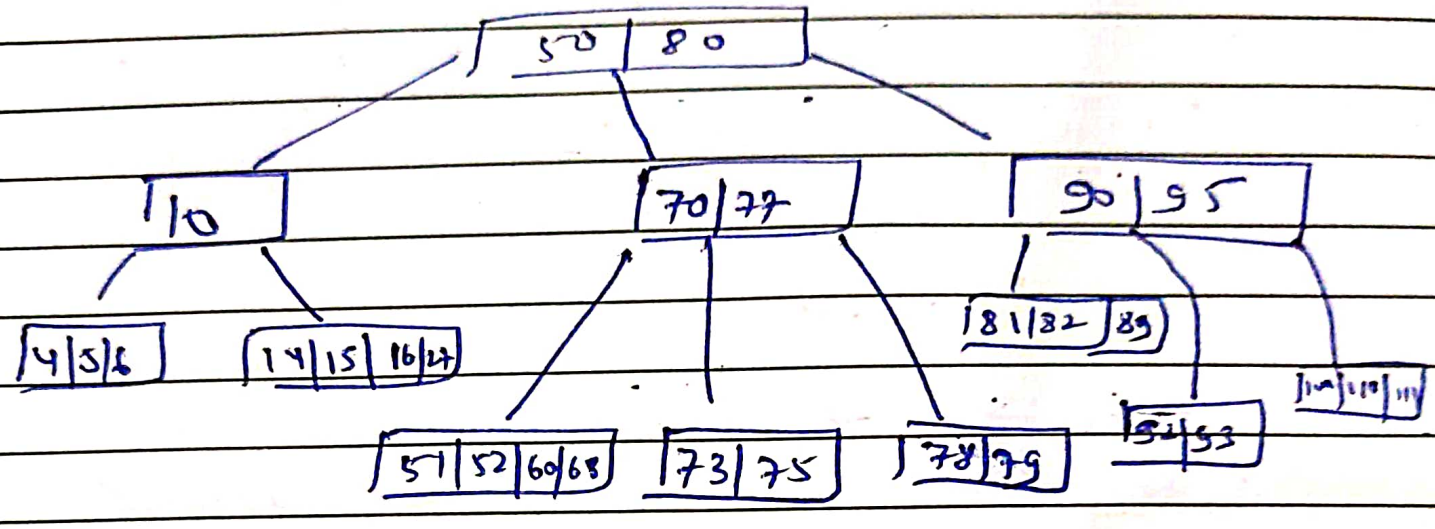


→ we cannot move 16 downwards as node with contain only one key.

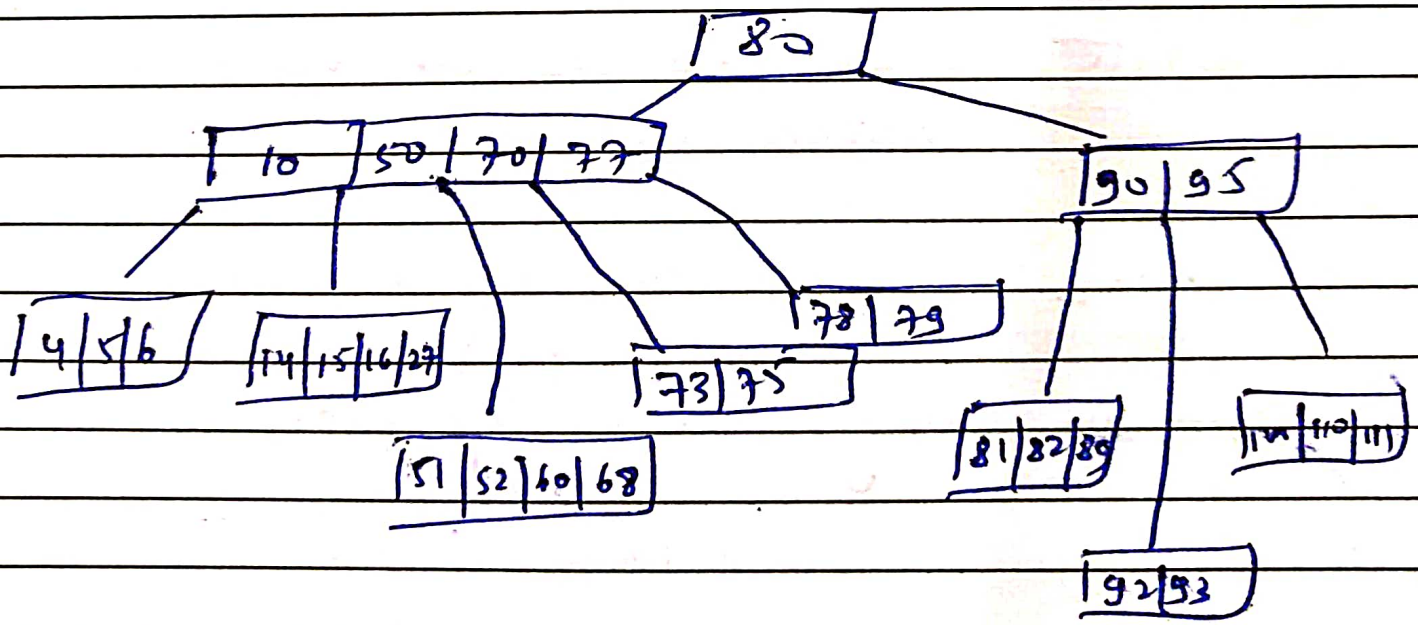
14 | 15 | 16 | ~~20~~ | 27

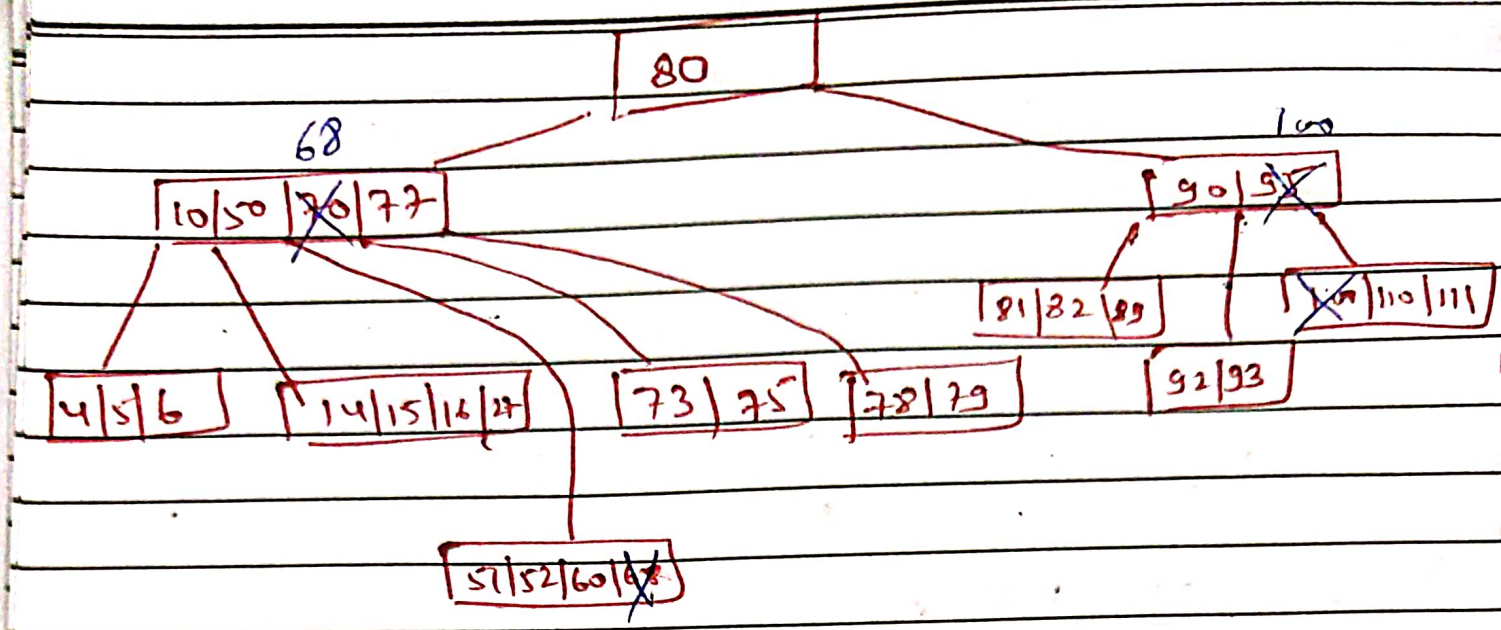


∴ 10 will ask from its sibling, it's sibling also contains min. keys ∴ merging will be done.



⇓



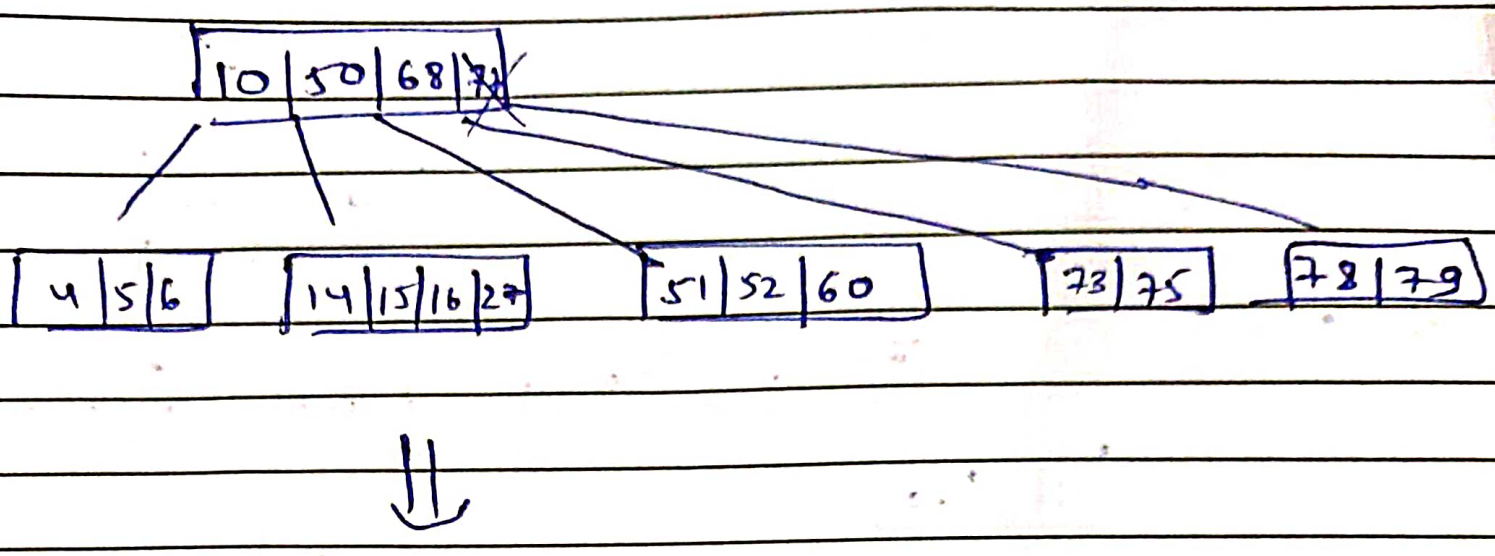


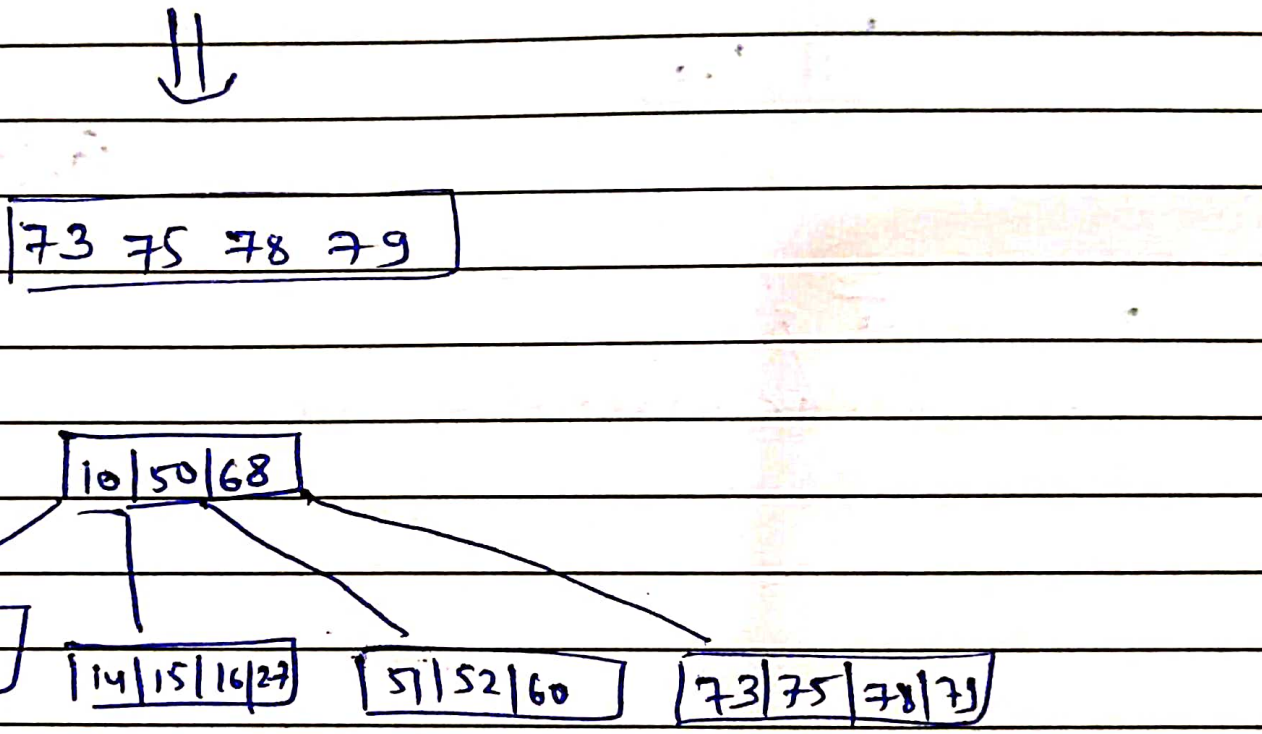
delete 70 → we can replace it with inorder predecessor or inorder successor whichever is having more than minimum no. of keys.

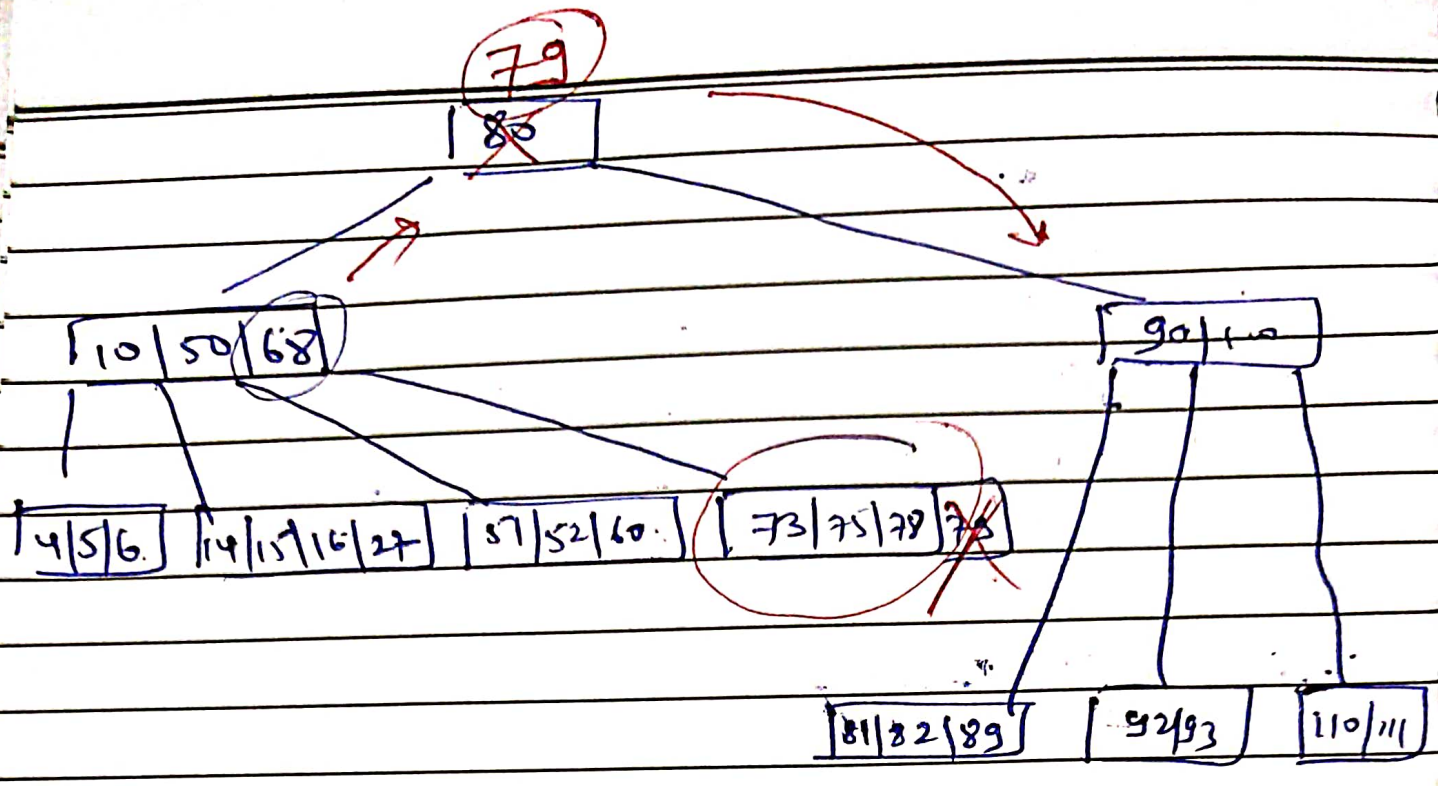
delete 95 → Now we can take from inorder successor.

delete 77 → inorder successor node & inorder predecessor node, both are having min. no. of nodes

∴ merging will take place.

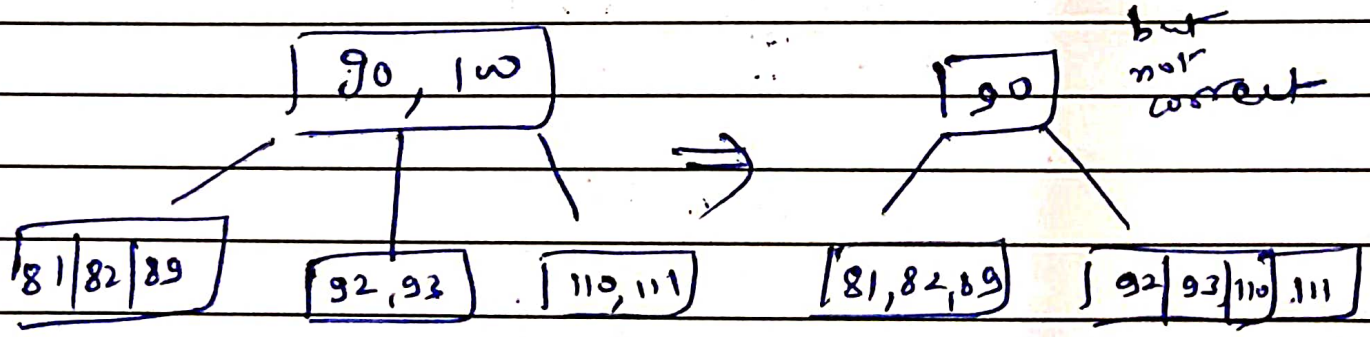

10 50 68 ~~79~~
4 5 6
14 15 16 27
51 52 60
73 75
78 79


73 75 78 79
10 50 68
4 5 6
14 15 16 27
51 52 60
73 75 78 79



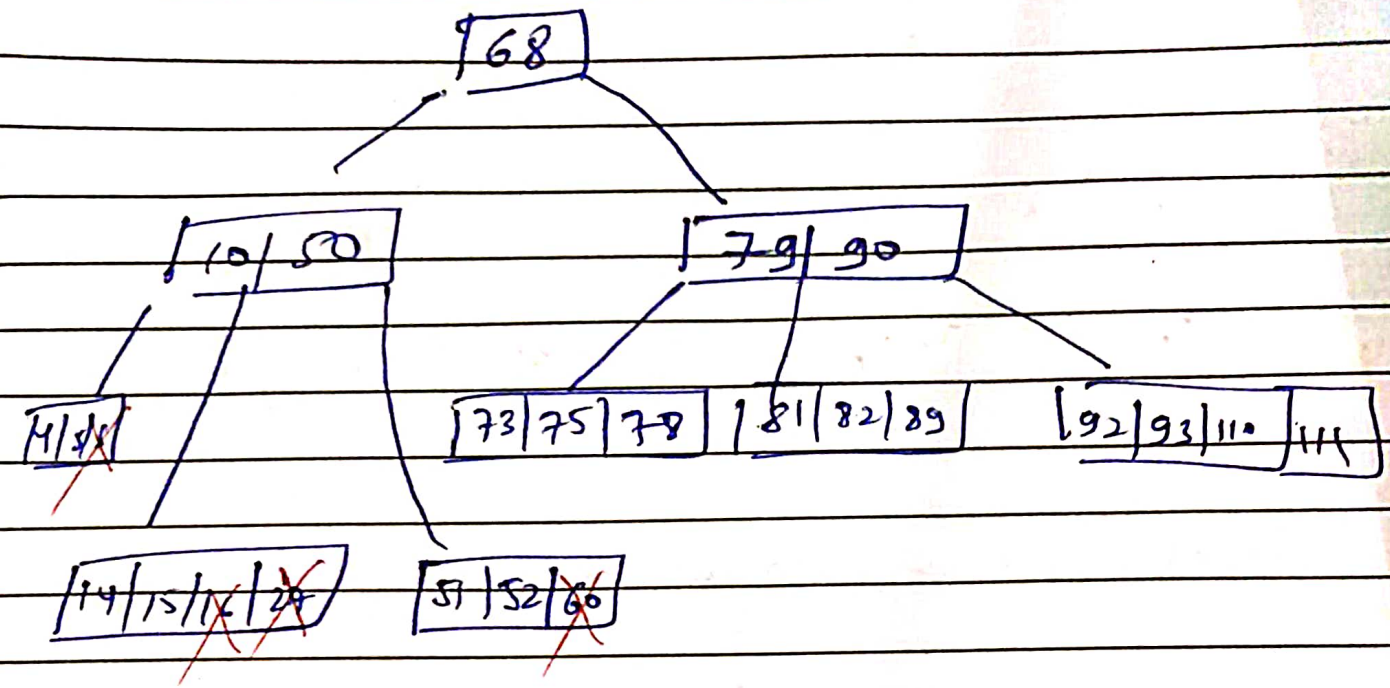
→ delete 80
check inorder predecessor i.e 79

→ delete 100



→ 90 will ask from its sibly

→ right part of 68 will also be transferred

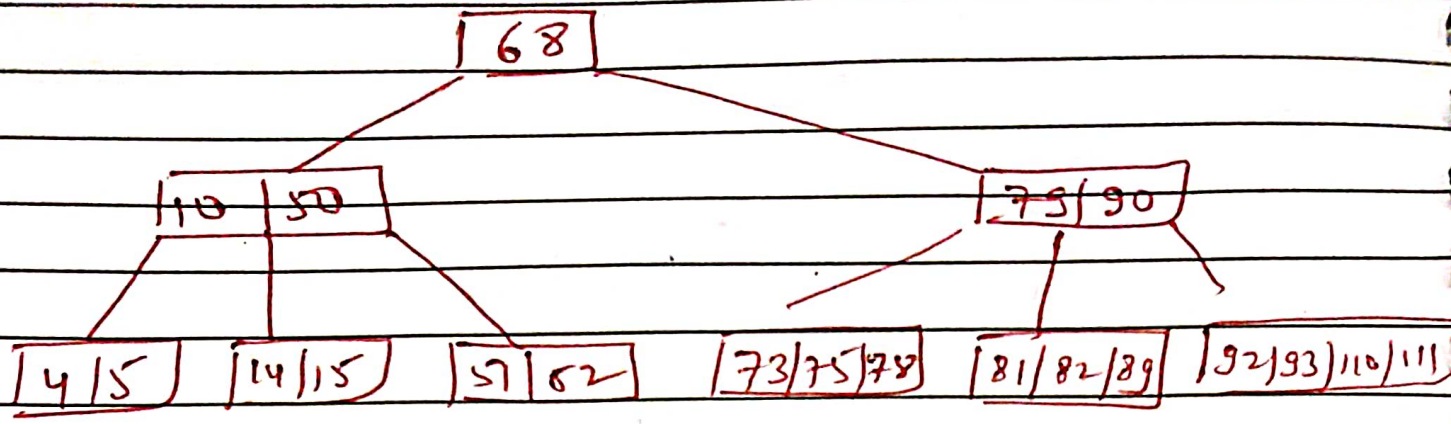


delete 6 → simply delete it

delete 27 → simply delete it

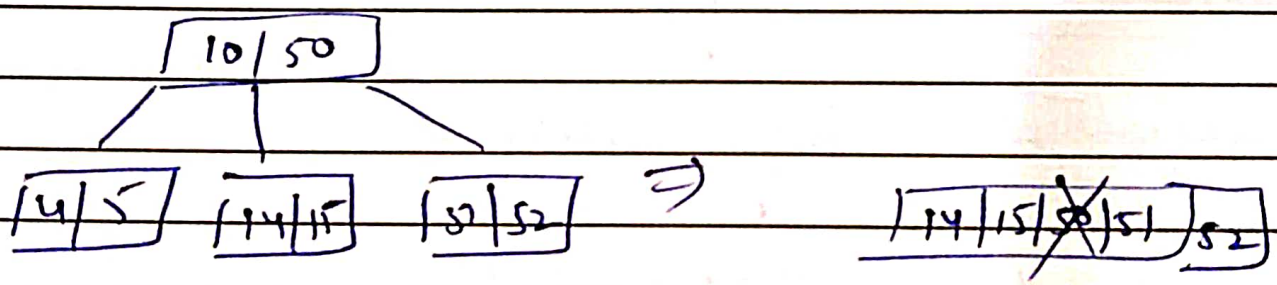
delete 60 → simply delete it

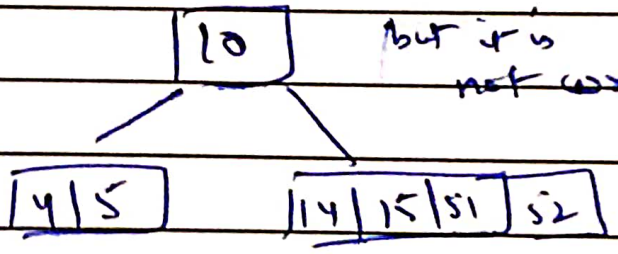
delete 16 → simply delete it



delete 50 → both inorder & inorder are
successor predecessor

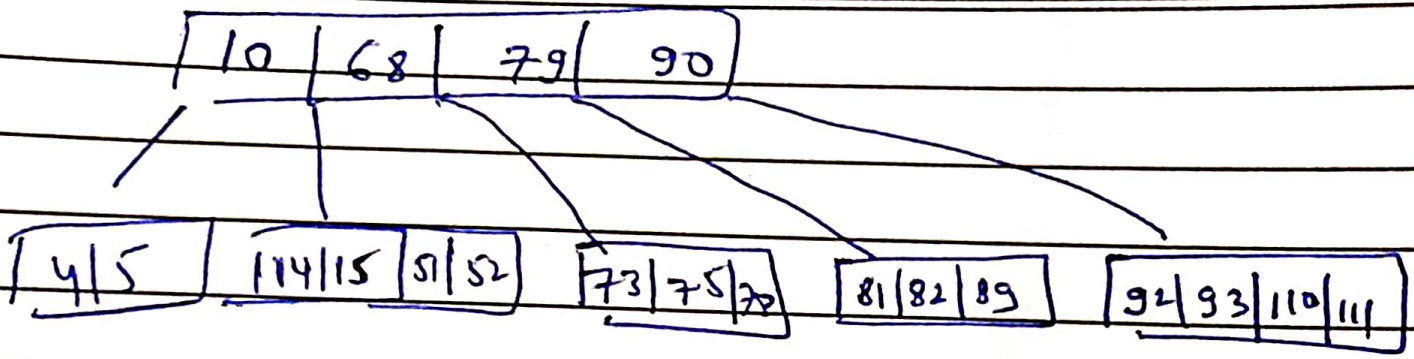
having children equal to minimum no.
∴ merging.





but it is not correct

→ will ask from sibly but sibly is also not having more child


10 | 68 | 79 | 90
4 | 5
14 | 15
51 | 52
73 | 75 | 78
81 | 82 | 89
92 | 93 | 110 | 111